# Meeting 2

Tuesday, April 28, 2026 3:31 PM

**Intro**

Last meeting, we agreed on the general plan:

- First run explores and remembers (DFS)
- Later runs use the memory to make better moves
- BFS can help find a shorter known path
- If BFS is not ready, we use a backup movement strategy

Today, we need to make sure everyone is coding pieces that actually connect

**New information:**

- Source code he provided is a courtesy so that we can model our algorithm. Ignore the bugs in his code. Can't use anything on his code to help us so just focus on our algorithm.
- Can email him for libraries we need like stack, etc.

**Progress Check**

To everyone:

- ~~Did you code anything?~~
- ~~Does it compile?~~
- ~~What file did you change?~~
- ~~Did you push it?~~

**Data Abstraction**

- **What has everyone thought of?**
- Especially Joel

- Static vector of points to map the maze
- Static end variable
-

We need to agree on the following:

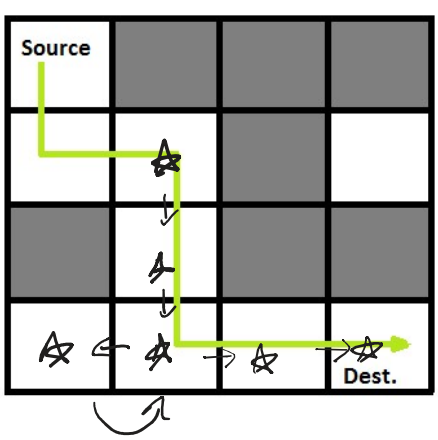
1. ~~How we track the racer's current position~~
    a. ~~Its already done for us, car.getLocation~~
2. How we track visited cells
3. How we track known open paths
4. How we track known walls
5. How we store the BFS path
6. How BFS knows what map data to read
7. How exploration writes data that BFS can use

~~**Backup Plan (in case some of us have coded already)**~~

- ~~Then we need to compare code and decide which to use~~

Once we reach the end, we would get reset to 0,0

To make sure we don't confuse when we start and when we found the end, we can also check the flag variables



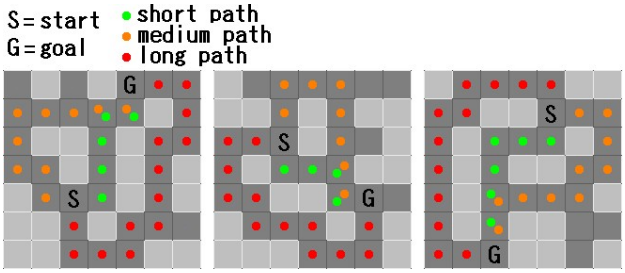

| | | | |
|---|---|---|---|
| 0 | 0 | 0 | 0 |
| 0 | 1 | 0 | 0 |
| 0 | 1 | 0 | 0 |
| 0 | 1 | 1 | 1 |

0 = unexplored

1 = visited

2 = ...

Current and previous location variable,




Might wanna use dequeue, value that maps